Google Cloud Shell Exercise: Manipulating Files and Directories

Marcus Birkenkrahe

February 2, 2026

Overview

In this exercise you will practice basic file and directory manipulation in **Google Cloud Shell**.

Rules:

- Type every command yourself.
- After each task group, verify your result using the listed check commands.
- Keep your work inside a dedicated directory named `playground`.

Setup

1. Open **Google Cloud Shell**.
2. Confirm you are in your home directory:
   - Use `pwd` and verify the path looks like `/home/<username>`.
3. You can always return home with:
   - `cd ~`

Building a playground

1. Change to your home directory.
2. Print your working directory.

3. Make a directory named `playground`.
4. Switch on the verbose option `-v` for commands that support it.
5. Check that it worked using `ls`, a pipeline, and `grep`.

```
cd ~
pwd
mkdir -v playground
ls -l | grep playground
```

Creating directories

1. Change directory to `playground` (do this at the start of each code block in this exercise).
2. Print your working directory.
3. Make two directories: `dir1` and `dir2`.
4. Switch on verbose output.
5. Check that it worked using `ls` and `grep`.

```
cd ~/playground
pwd
mkdir -v dir1 dir2
ls -l | grep -E 'dir[12]'
```

Copying files

1. Copy `/etc/passwd` into the current working directory (`playground`).
2. Switch on verbose output for the copy command.
3. Check that it worked using `ls` and a pattern match.

```
cd ~/playground
cp -v /etc/passwd .
ls -l | grep -E 'pass..'
```

Moving and renaming files

Rename the file

1. Rename `passwd` to `fun` (use verbose output).
2. Check that it worked using a wildcard pattern.

```
cd ~/playground
mv -v passwd fun
ls -l | grep -E 'fun'
```

Move the file into dir1 and inspect

1. Move `fun` into `dir1`.
2. Check with `ls -l`.

```
cd ~/playground
mv -v fun dir1/
ls -l dir1
```

Move fun from dir1 to dir2 in one command

1. Move `fun` from `dir1` to `dir2` using a single command.
2. Check with `ls -l`.

```
cd ~/playground
mv -v dir1/fun dir2/
ls -l dir2
```

Move fun back to playground

1. Move `fun` back to the current working directory.
2. Check with `ls -l`.

```
cd ~/playground
mv -v dir2/fun .
ls -l
```

Move dir1 into dir2 (directory move semantics)

1. Move `fun` into `dir1` again.
2. Move directory `dir1` into directory `dir2`.
3. Confirm that the file is now under `dir2/dir1/`.

```
cd ~/playground
mv -v fun dir1/
mv -v dir1 dir2/
ls -l dir2
ls -l dir2/dir1
```

Note:

- `dir1` was moved **into** `dir2` because `dir2` already existed.
- If `dir2` had not existed, `dir1` would have been **renamed** to `dir2`.

Put everything back (always use -v)

Goal state at the end of this subsection:

- `~/playground/dir1` exists (directly under `playground`)
- `~/playground/fun` exists (directly under `playground`)
- `dir2` exists and is empty

```
cd ~/playground
mv -v dir2/dir1 .
mv -v dir1/fun .
ls -l
ls -l dir1
ls -l dir2
```

Creating hard links

1. Create a hard link named `fun-hard` to `fun` in the current directory.
2. Create a hard link named `fun-hard` to `fun` in `dir1`.
3. Create a hard link named `fun-hard` to `fun` in `dir2`.

4. Switch on the verbose option for `ln`.

5. Confirm with `ls -l` for each location.

```
cd ~/playground
ln -v fun fun-hard
ln -v fun dir1/fun-hard
ln -v fun dir2/fun-hard
ls -l .
ls -l dir1
ls -l dir2
```

The number in the `ls -l` listing (link count) shows how many hard links exist for the file.

Show that `fun` and the hard links refer to the same inode:

```
cd ~/playground
ls -li fun fun-hard dir1/fun-hard dir2/fun-hard
```

Creating symbolic links

1. Create a symbolic link named `fun-sym` to `fun` in the current directory.

2. Create a symbolic link named `fun-sym` to `fun` in `dir1`.

3. Create a symbolic link named `fun-sym` to `fun` in `dir2`.

4. Switch on verbose output for `ln`.

5. Confirm with `ls -l`.

```
cd ~/playground
ln -vs fun fun-sym
ln -vs fun dir1/fun-sym
ln -vs fun dir2/fun-sym
ls -l .
ls -l dir1
ls -l dir2
```

Create a symbolic link to a directory:

- Create `dir1-sym` pointing to `dir1` in the current directory.

```
cd ~/playground
ln -vs dir1 dir1-sym
ls -l .
```

Optional check:

- Compare inode values of `dir1` and `dir1-sym` (they should differ; the symlink is its own file).

```
cd ~/playground
ls -ldi dir1 dir1-sym
```

Removing files and directories

Remove one hard link

1. Remove the hard link `fun-hard` in the current directory (use verbose output).
2. Confirm with `ls -l`.

```
cd ~/playground
rm -v fun-hard
ls -l .
```

Create and remove a test file

1. Create a file named `y` containing the single character `y`.
2. Confirm it exists and has content.

```
cd ~/playground
echo "y" > y
ls -l y
cat y
```

Remove the original file fun

1. Remove `fun` (use verbose output).
2. Confirm with `ls -l`.

```
cd ~/playground
rm -v fun
ls -l .
```

Now check what happens to the symbolic link:

- Run `cat fun-sym`
- You should get an error like:
  - `fun-sym:  No such file or directory`

```
cd ~/playground
cat fun-sym
```

Make sure you understand:

- A symbolic link stores a path to a target.
- If the target is removed, the symlink still exists, but it becomes a **broken symbolic link**.

Remove the symbolic links (use verbose output)

Remove all `fun-sym` links and the directory symlink.

```
cd ~/playground
rm -v fun-sym dir1/fun-sym dir2/fun-sym dir1-sym
ls -l .
ls -l dir1
ls -l dir2
```

Remove the playground

Go to your home directory and remove `playground` recursively with verbose output.

```
cd ~
rm -vr playground
ls -l
```

Command summary

Fill in the table.

COMMAND	MEANING	EXAMPLE
cd	Change current directory	cd ~/playground
pwd	Print working directory	pwd
mkdir	Create a new directory	mkdir dir1
echo	Print text to standard output	echo "hi"
mv -v	Move or rename files, verbose	mv -v a b
rm -vr	Remove files/directories	rm -vr playground
ln -vs	Create symbolic link, verbose	ln -vs a b
ls -l	List files with details	ls -l